

DAA Theory Notes — Unit III (Part 2)

Dynamic Programming

UNIT III — Dynamic Programming

1. Dynamic Programming — Introduction

1. Dynamic Programming is an algorithm design method applied to **optimization problems** where the solution can be viewed as the result of a **sequence of decisions**.
 2. It is a **bottom-up approach** — smaller subproblems are solved first and their results are stored in a table, which are then used to build solutions to larger problems.
 3. Unlike Divide and Conquer which solves independent subproblems, DP solves **overlapping subproblems** and avoids recomputation by storing previously computed results.
 4. Unlike Greedy which makes only a locally optimal decision, DP considers **all possible decisions** at each stage and picks the globally optimal one.
 5. To apply Dynamic Programming, the problem must satisfy the **Principle of Optimality** — the overall optimal solution must contain optimal solutions to all its subproblems.
-

2. Principle of Optimality

1. The Principle of Optimality states that in an **optimal sequence of decisions**, every subsequence of those decisions must also be individually optimal.
 2. Formally: if $D_1, D_2, \dots, D_n$ is an optimal sequence of decisions, then the last k decisions (for any $1 < k < n$) must also form an optimal sub-sequence.
 3. **Example — Shortest Path:** If the path $i \rightarrow i_1 \rightarrow i_2 \rightarrow j$ is the shortest path from i to j , then $i_1 \rightarrow i_2 \rightarrow j$ must be the shortest path from i_1 to j . If it were not, we could replace it with a shorter path and get a contradiction.
 4. A problem that satisfies the Principle of Optimality can always be **represented as a multistage graph** and solved using Dynamic Programming.
 5. Problems that do NOT satisfy the Principle of Optimality cannot be solved using DP — for example, the longest simple path problem in a graph with cycles violates this principle.
-

3. Memoization (Overlapping Subproblems)

1. Memoization is the technique of **storing the results of already-solved subproblems** in a table so they are never recomputed from scratch again.
2. Without memoization, a recursive solution recomputes the same subproblem results exponentially many times, leading to very poor time complexity.

3. **Example — Fibonacci:** Computing $f(5)$ recursively without memoization recalculates $f(3)$ twice, $f(2)$ three times, and so on. With a table, each value is computed exactly once.
 4. The table is typically a **1D or 2D array** depending on how many parameters define the subproblem state — for example, Knapsack uses $B[i][w]$ indexed by item number and weight.
 5. Memoization converts an exponential recursive solution into a **polynomial time DP solution** by ensuring each unique subproblem is solved only one time.
-

4. Dynamic Programming vs Divide and Conquer

Feature	Dynamic Programming	Divide and Conquer
Approach	Bottom-up — solves small subproblems first and builds up	Top-down — breaks the large problem into subproblems recursively
Subproblems	Overlapping — same subproblems appear and are reused multiple times	Independent — subproblems do not overlap or share results
Storage	Uses a table to store and reuse previously computed subproblem results	Does not store subproblem results; recomputes if needed
Optimality	Guarantees a globally optimal solution using the Principle of Optimality	Always correct but does not specifically target optimization
Examples	0/1 Knapsack, LCS, TSP, Multistage Graph	Merge Sort, Quick Sort, Binary Search

5. Dynamic Programming vs Greedy

Feature	Dynamic Programming	Greedy
Decision making	Considers all possible decisions at each stage and picks the best globally	Makes only the single locally best decision at each step
Backtracking	Can reconsider all choices through table lookups and optimal substructure	Never reconsiders a decision once made
Optimality	Always produces the globally optimal solution for problems with optimal substructure	Produces globally optimal solution only for specific problems like Fractional Knapsack
Complexity	Generally higher time and space complexity due to table construction	Faster — usually $O(n \log n)$ due to sorting only

Feature	Dynamic Programming	Greedy
Examples	0/1 Knapsack, LCS, TSP	Fractional Knapsack, Huffman Encoding, Job Sequencing

6. 0/1 Knapsack Problem — Theory

1. Given n items each with weight w_i and profit p_i , and a knapsack of capacity M , the goal is to **maximize total profit** where each item is either taken completely ($x_i = 1$) or not taken at all ($x_i = 0$) — no fractions allowed.
2. It cannot be solved greedily because the optimal solution for items $\{1, 2, \dots, k+1\}$ may **not contain** the optimal solution for items $\{1, 2, \dots, k\}$ — so greedy choice may lead to a wrong answer.
3. The DP table $B[i][w]$ represents the **maximum profit achievable** using the first i items with a total weight limit of w .
4. **Recurrence Relation:**
 - If $w_i > w$: $B[i][w] = B[i-1][w]$ — item i is too heavy, so exclude it
 - If $w_i \leq w$: $B[i][w] = \max \{ B[i-1][w], B[i-1][w-w_i] + v_i \}$ — choose the better of including or excluding item i
5. The final answer is $B[n][M]$ and to find which items are selected, we **trace back** through the table — if $B[i][w] \neq B[i-1][w]$ then item i is included and we move to $B[i-1][w-w_i]$, otherwise item i is excluded.
6. **Time Complexity:** $O(n \times W)$ — filling an $n \times W$ table. **Space Complexity:** $O(n \times W)$ for the table.

7. 0/1 Knapsack — Algorithm (For Exam)

```

Algorithm Knapsack(n, W, w[], v[])
{
    for w = 0 to W:
        B[0][w] = 0          // Initialize row 0 to 0

    for i = 1 to n:
        B[i][0] = 0          // Initialize column 0 to 0

    for i = 1 to n:
        for w = 0 to W:
            if (w[i] <= w):
                if (v[i] + B[i-1][w-w[i]] > B[i-1][w]):
                    B[i][w] = v[i] + B[i-1][w-w[i]]
            else:

```

```

        B[i][w] = B[i-1][w]
    else:
        B[i][w] = B[i-1][w]

    return B[n][W]
}

```

8. Longest Common Subsequence (LCS) — Theory

1. A **subsequence** is a sequence derived from another sequence by deleting some elements without changing the relative order of the remaining elements — the elements do not need to be contiguous.
 2. The LCS problem: given two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$, find the **longest sequence Z that appears as a subsequence of both X and Y**.
 3. **Example:** For $X = \langle A, B, C, B, D, A, B \rangle$ and $Y = \langle B, D, C, A, B, A \rangle$, the LCS is $\langle B, C, B, A \rangle$ or $\langle B, D, A, B \rangle$, both of length 4.
 4. **Recurrence Relation** using table $C[i][j]$ = length of LCS of $X[1..i]$ and $Y[1..j]$:
 - If $i = 0$ or $j = 0$: $C[i][j] = 0$ — empty sequence base case
 - If $X[i] = Y[j]$: $C[i][j] = C[i-1][j-1] + 1$ — characters match, extend LCS
 - If $X[i] \neq Y[j]$: $C[i][j] = \max(C[i-1][j], C[i][j-1])$ — take the better of excluding either character
 5. **Time Complexity:** $O(m \times n)$ where m and n are the lengths of the two sequences. **Space Complexity:** $O(m \times n)$ for the table.
-

9. LCS — Traceback to Print the Sequence

1. After filling the C table, a separate **direction table $b[i][j]$** is maintained to record which case was used — diagonal ($\nearrow$), up ($\uparrow$), or left ($\leftarrow$).
2. A diagonal arrow ($\nearrow$) means $X[i] = Y[j]$ and this character is **part of the LCS**.
3. An up arrow ($\uparrow$) means the LCS was inherited from $C[i-1][j]$, so we move up and the current character is **not in the LCS**.
4. A left arrow ($\leftarrow$) means the LCS was inherited from $C[i][j-1]$, so we move left and the current character is **not in the LCS**.
5. The traceback starts from $b[m][n]$ and follows arrows recursively until $i = 0$ or $j = 0$ — characters are printed only when a diagonal arrow is encountered.

```

Algorithm PRINT-LCS(b, X, i, j)
{
    if i = 0 or j = 0: return
    if b[i][j] = '↖':
        PRINT-LCS(b, X, i-1, j-1)
        print X[i]
    else if b[i][j] = '↑':
        PRINT-LCS(b, X, i-1, j)
    else:
        PRINT-LCS(b, X, i, j-1)
}

```

10. Travelling Salesman Problem (TSP) — Theory

1. In the TSP, a salesman must visit **all n cities exactly once** and return to the starting city such that the **total distance (cost) of the tour is minimized**.
 2. A brute-force approach would check all $(n-1)!$ possible tours, which is computationally infeasible for large n — DP reduces this significantly.
 3. The DP formulation uses $g(i, S)$ = minimum cost path starting from city i , passing through all cities in set S exactly once, and returning to city 1 (the starting city).
 4. **Recurrence Relation:**
 - $g(i, \emptyset) = c(i, 1)$ — cost of going directly back to city 1 with no intermediate cities
 - $g(i, S) = \min \text{ over all } j \text{ in } S \text{ of } \{ c(i, j) + g(j, S - \{j\}) \}$ — go to some city j first, then optimally visit the rest
 5. The final optimal tour cost is $f = g(1, \{2, 3, \dots, n\}) = \min \text{ over all } k \neq 1 \text{ of } \{ c(1, k) + g(k, \{2, \dots, n\} - \{k\}) \}$.
 6. **Time Complexity:** $O(n^2 \times 2^n)$ — much better than $O(n!)$ brute force but still exponential, so DP only helps for moderate n .
-

11. TSP — Step-by-Step DP Procedure (For Exam)

1. **Step 1 — Base Case (0-element sets):** Compute $g(i, \emptyset) = c(i, 1)$ for all cities $i \neq 1$ — this is the direct cost of each city returning straight to the start.
2. **Step 2 — 1-element sets:** For each city i and each single-city set $\{j\}$, compute $g(i, \{j\}) = c(i, j) + g(j, \emptyset)$ — cost of going to j and then directly to city 1.
3. **Step 3 — 2-element sets:** For each pair $\{j, k\}$, compute $g(i, \{j, k\}) = \min \{ c(i, j) + g(j, \{k\}), c(i, k) + g(k, \{j\}) \}$ — choose the cheaper intermediate city to visit first.
4. **Step 4 — Continue** until all subsets of cities are evaluated, then compute $f = g(1, \{2, 3, \dots, n\})$ for the final optimal tour cost.

5. **Step 5 — Traceback:** Identify which city was chosen at each step to reconstruct the actual optimal tour path (e.g., $1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1$).
-

12. Multistage Graph Problem — Theory

1. A **multistage graph** $G = (V, E)$ is a directed weighted graph where vertices are partitioned into k stages, and edges only go **from one stage to the next** — never backwards.
 2. The problem is to find the **minimum cost path from the source (stage 1) to the sink (stage k)**, passing through exactly one vertex per stage.
 3. The **Greedy method fails** for multistage graphs because locally choosing the cheapest next edge does not guarantee the globally cheapest overall path.
 4. Two DP approaches exist — **Forward Approach** (starts from sink, works backwards using $fcost[j]$) and **Backward Approach** (starts from source, works forwards using $bcost[j]$).
 5. **Time Complexity:** $\Theta(|V| + |E|)$ when the graph is represented using an adjacency list, making it very efficient for large sparse multistage graphs.
-

13. Multistage Graph — Forward Approach ($fcost$)

1. In the forward approach, we define $fcost[j]$ = minimum cost path from vertex j to the sink (last vertex n).
2. **Base case:** $fcost[n] = 0$ — the cost from the sink to itself is zero.
3. For every other vertex j (processed from $n-1$ down to 1), $fcost[j] = \min$ over all edges (j,r) of $\{c[j][r] + fcost[r]\}$ — go to the best next vertex r and add its future cost.
4. Along with $fcost$, we maintain $d[j]$ = the next vertex r chosen for vertex j , so we can reconstruct the actual path at the end.
5. The optimal path is recovered using $d[]$ — start from $p[1] = 1$ (source), then follow $p[j] = d[p[j-1]]$ for each stage until reaching the sink.

```
Algorithm Fgraph(G, k, n, p)
{
    fcost[n] = 0.0;
    for j = n-1 to 1 step -1:
        find r such that <j,r> is an edge and c[j][r] + fcost[r] is minimum
        fcost[j] = c[j][r] + fcost[r]
        d[j] = r
    p[1] = 1;  p[k] = n;
    for j = 2 to k-1:
        p[j] = d[p[j-1]]
}
```

14. Multistage Graph — Backward Approach (bcost)

1. In the backward approach, we define **bcost[j]** = minimum cost path from the source (vertex 1) to vertex j.
 2. **Base case:** **bcost[1] = 0** — the cost from source to itself is zero.
 3. For every other vertex j (processed from 2 up to n), **bcost[j]** = min over all edges (r,j) of { **bcost[r] + c[r][j]** } — come from the best previous vertex r and add its cost.
 4. The backward approach uses **forward reasoning** — we reason about how we arrived at each vertex from the source.
 5. Both approaches yield the **same optimal cost** — the choice between them depends on which direction is easier to implement given the graph structure.
-

15. Forward vs Backward Approach — Key Difference

1. If the **recurrence is formulated using the Forward Approach** (working from source to sink), then the relations are **solved backwards** — starting from the last decision and working back to the first.
 2. If the **recurrence is formulated using the Backward Approach** (working from sink to source), then the relations are **solved forwards** — starting from the first decision and working to the last.
 3. The naming is a common point of confusion — **forward formulation = backward solution**, and **backward formulation = forward solution**.
 4. In the Fgraph algorithm, we compute fcost going **right to left** (n-1 down to 1) and then reconstruct path going **left to right** (stage 1 to stage k).
 5. In the Bgraph algorithm, we compute bcost going **left to right** (2 to n) and then reconstruct path going **right to left** (stage k-1 down to stage 2).
-

16. Summary — DP Algorithm Complexities

Algorithm	Time Complexity	Space Complexity
0/1 Knapsack (DP)	$O(n \times W)$	$O(n \times W)$
Longest Common Subsequence	$O(m \times n)$	$O(m \times n)$
Travelling Salesman Problem (DP)	$O(n^2 \times 2^n)$	$O(n \times 2^n)$
Multistage Graph (Forward/Backward)	$\Theta(V + E)$	$O(n)$

End of Unit III — Dynamic Programming Theory Notes All the best!